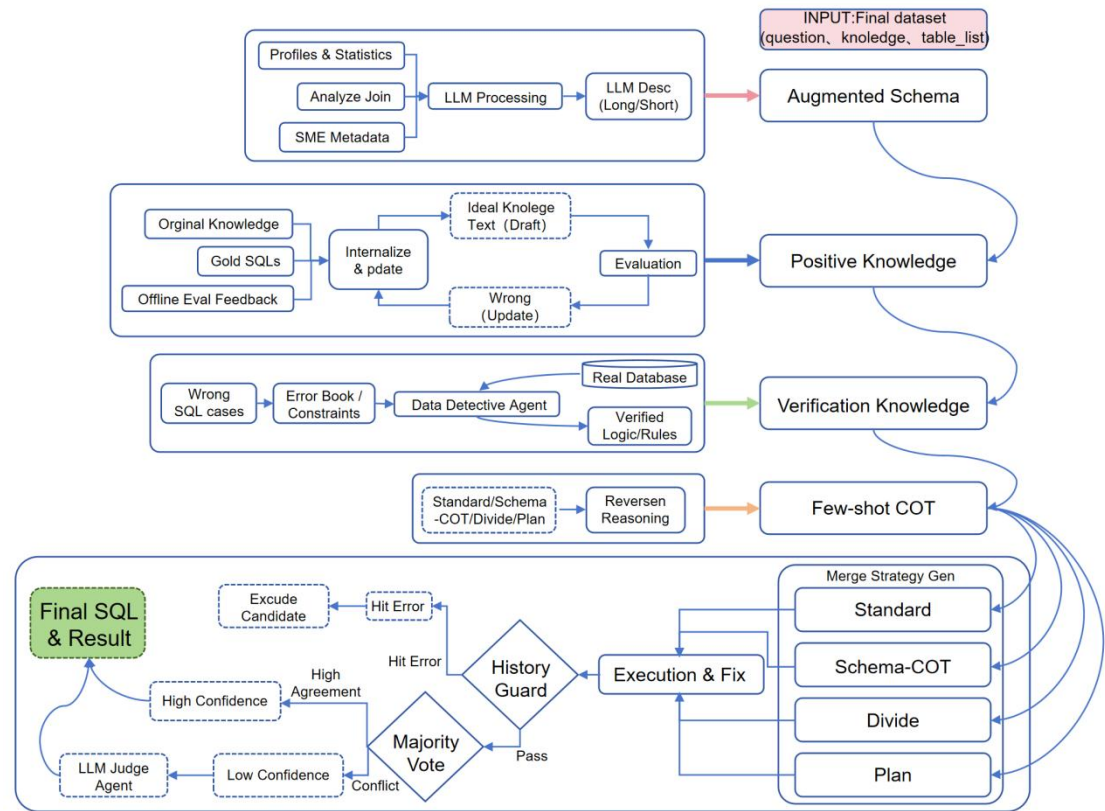


# 基于 Agentic Workflow 与闭环知识进化的 Text-to-SQL 架构

## 1.整体方案：



本方案旨在超越传统 Text-to-SQL 依赖静态提示词的范式，构建了一套集“数据驱动、知识闭环、多路博弈”于一体的复合架构。系统以 `final_dataset` 为输入，经由四层动态知识构建流程处理，最终输入多策略执行引擎。通过严格的执行校验与裁决机制，确保输出 SQL 的高准确率。

在进入核心推理阶段前，系统预先通过四个维度对原始数据进行深度处理，构建动态知识库：

**(1) Schema 增强(Augmented Schema)**针对原始 Schema 语义稀疏且缺乏业务上下文的问题，系统集成了数据剖析 (Profiles & Statistics)、连接关系分析 (Analyze Join) 及专家元数据 (SME Metadata)。利用大语言模型 (LLM) 对上述统计特征（如枚举值集合、数据分布特征、潜在连接键）进行综合建模，自动生成包含简述 (Short) 与详述 (Long) 的增强版 Schema 描述。该过程旨在将底层数据特征显性化，辅助模型准确理解数据表的真实结构与语义。

**(2) 正向知识闭环(Positive Knowledge Loop)**构建基于“生成-评估-修正”闭环的正向知识迭代体系。系统以标准 SQL 为基准，通过反向解析提取业务逻辑描述 (Ideal Knowledge Text)。该模块引入自动化评估机制：将提取的逻辑作为上下文输入模型进行二次推理，通过执行结果的一致性校验其有效性。仅当逻辑能正确引导代码生成时，才将其固化为正向知识 (Positive Knowledge)；验证失败的逻辑触发回流修正。这一机制保证了知识库内容的准确性与实战可用性。

**(3) 验证与负向约束(Verification Knowledge)**建立基于“历史错题约束+主

动数据探查”的双重验证机制。一方面，利用历史评测中的错误案例构建错误集（Error Book），作为推理过程中的负向约束条件，防止重复错误；另一方面，部署数据探查代理（Data Detective Agent），针对模糊的业务假设（如默认字段值、隐性过滤条件），主动向真实数据库发起探测性查询（Active Probing）。通过真实数据反馈验证假设的正确性，并将验证后的规则转化为验证型知识（Verification Knowledge），以弥补模型对隐性业务规则认知的不足。

**（4）规范化推理反推(Few-shot CoT)**采用“规范化思维链构建+多维混合检索”策略，替代传统的问答对示例。首先，利用反向工程技术将标准 SQL 解构为包含执行计划（Plan）、任务拆解（Divide）与思维链（CoT）的规范化推理步骤，引导模型学习解题逻辑。其次，构建三维检索机制：基于脱敏问题（Masked Question）匹配结构相似性，基于知识逻辑（Knowledge Logic）匹配业务相似性，基于表上下文（Table Context）匹配环境相似性。该策略旨在精准检索出在逻辑与结构上最优匹配的参考样本。

在完成动态知识构建后，系统正式进入核心推理与执行阶段，承接前序生成的增强 Schema 与 Few-shot CoT 上下文，通过“多路竞争-执行验证-智能裁决”的三层架构，确保最终输出 SQL 的准确性与鲁棒性：

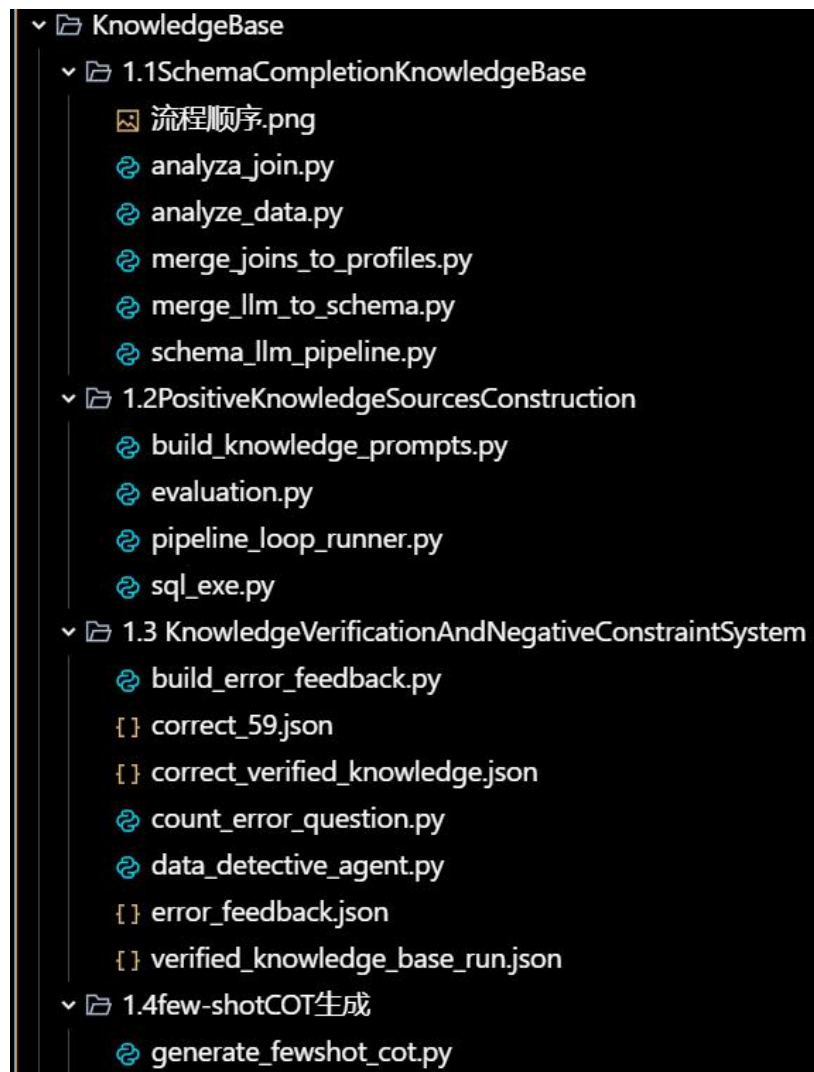
**（1）多路差异化生成(Multi-Strategy Generation)** 为克服单一推理路径的幻觉与局限性，系统并行调度四个具有差异化思维模式的 Agent 进行候选生成：Standard（基准直觉）、Schema-COT（专注于列名映射与值对齐）、Divide（针对复杂嵌套逻辑的分治拆解）以及 Plan（模拟数据库优化器的路径规划）。通过输入侧的视图扰动与推理侧的策略互补，最大化解题思路的多样性，确保在面对不同复杂度的问题时均有适配的解题路径。

**（2）执行修复与历史防御(Execution,Fix&History Guard)** 旨在构建鲁棒的执行环境与错误免疫机制。生成的候选 SQL 首先进入沙箱环境执行，若遇语法或逻辑错误，立即触发自动修复机制（Execution&Fix）。随后，所有执行成功的 SQL 必须通过 History Guard（历史防御）检查：系统计算执行结果的 Hash 值，并与“错题本”中的已知错误记录进行比对。一旦命中历史错误（Hit Error），该候选方案将被直接剔除，从而在物理层面阻断重复错误的发生。

**（3）一致性投票与智能裁决(Consistency Voting&Arbitration)** 通过防御检查的候选方案进入最终决策阶段。系统采用 Majority Vote（多数一致性投票）机制：若存在高一致性的结果集（High Agreement），则判定为高置信度直接输出；若各 Agent 结果存在分歧（Conflict），则触发 LLM Judge Agent 介入。裁判代理基于业务逻辑规则对不同结果进行深度辨析与校验，打破僵局，最终选定最优解作为 Final SQL，实现准确率与稳定性的双重保障。

## 2. 知识库构建与注入 (KnowledgeBase)

文件树：



### 2.1 SchemaCompletionKnowledgeBase

表/列级剖析、连接键候选、LLM 摘要与校验

#### 设计初衷：

在真实的工业级数据库中，我们面临着严重的“语义鸿沟”问题：

**原始描述匮乏：**Schema 中的字段注释极其简短（如仅标注“金额”、“日期”），甚至缺失。

**业务语意模糊：**面对动辄几十上百列的宽表，大模型很难仅凭列名（如 f\_01,status\_cd）区分其具体的业务作用（是支付状态？还是物流状态？）。

**数据特征未知：**模型看不见真实数据，容易产生“幻觉”，例如对“YYYYMMDD”格式的整数列使用日期函数，或对非枚举列进行枚举查询。

#### 解决方案：

采用“深度数据剖析+LLM 逻辑增强”策略，为每个字段自动生成用于快速检索的“简述”和辅助推理的“详述”。我们首先摒弃盲目猜测，通过统计学扫描精准提取数据分布与高频枚举值，并利用 **MinHash 算法配合真实 SQL 执行**来挖掘并物理验证隐性连接键，彻底解决无外键关联难题。最终，LLM 化身“资深数据分析师”，融合上述硬核剖析报告（如时间范围、表更新频率）与原始元数据，综合推导出包含 **Deep Stats** 的增强版 Schema，实现了从“猜测元数据”到“透视真数据”的质变。

### ***analyzer\_join.py:***

对 schema.json 中每张表的每个字段做数据剖析，生成每表一份 JSON 报告到 profiling\_output\_per\_table

### ***analyzer\_join.py:***

基于各列的 MinHash 签名在两表之间寻找高相似连接键候选，并用实时数据库 JOIN 做验证；输出到 join\_candidates\_verified.json

### ***merge\_joins\_to\_profiles.py:***

把 join\_candidates\_verified.json 合并回每表剖析报告，为字段增加 potential\_links（潜在连接信息），并去除 minhash\_signature，输出到 profiling\_output\_merged

### ***merge\_llm\_to\_schema.py:***

将 profiling\_output\_merged 中字段/表的 LLM 描述合并回 schema.json，并按报告剔除全空字段，生成 schema\_all.json，并同步到 run/agent/schema.json。

### ***schema\_llm\_pipeline.py:***

数据库增强：为每列补 deep\_stats（时间字段的范围/计数、低基数枚举全集）和为整表补 verification\_stats（近若干天覆盖率推断快照/增量类型）。

## **2.2 PositiveKnowledgeSourcesConstruction**

离线评测、错误题收集、基于正误 SQL 与知识为每道题目生成的一段高度凝练的业务与数据逻辑指导文本（Ideal Knowledge Text），用于指导后续 SQL 生成与修复，避免重复犯错

### **核心痛点与设计初衷：**

在复杂的 Text-to-SQL 任务中，拥有标准答案（Gold SQL）并不等于拥有了解决问题的能力。单纯的 Few-shot 示例往往只能让模型模仿代码的“形”，而难以领会业务的“神”（即深层的逻辑约束）。模型常因缺乏对隐性业务规则（如特定的统计口径、隐藏的过滤条件）的理解，导致在面对类似问题时重复犯错。

### **解决方案：**

本模块构建了一套“提取-验证-内化”的闭环演进系统。首先，利用

`build_knowledge_prompts.py` 驱动模型扮演“业务专家”，对标准答案（Gold SQL）进行深度反向解码，将代码中的隐性约束（如特定过滤条件、表关联逻辑）蒸馏为显性化的“黄金业务逻辑（Ideal Knowledge Text）”。随后，为了确保知识的准确性，系统通过 `pipeline_loop_runner.py` 启动自动化实战校验：强制 Agent 加载新生成的业务理解重新解题，生成的 SQL 经由 `sql_exe.py` 在真实数据库中执行并进行数值标准化处理，最后通过 `evaluation.py` 与标准答案进行严格的结果级比对（Result-based Evaluation）。只有那些能指导模型成功复现正确结果的逻辑，才会被判定为“有效知识”并正式合并入库，从而实现将静态答案转化为经得起实战考验的动态真理。

### ***build\_knowledge\_prompts.py:***

围绕每个错题 `sql_id`，汇总“题目问题、表结构精简、正确 SQL、错误 SQL 样本、旧知识、已验证知识、通用知识”等材料，调用 Gemini 生成凝练的“黄金知识文本”，并写入与合并到知识库。

### ***pipeline\_loop\_runner.py***

多轮自动化跑整个 pipeline，直到关注集合中的错题清零或达到轮次上限。每轮包含：Agent 生成→SQL 执行→评测→知识构建

### ***sql\_exe.py***

通过 pymysql 执行 SQL（批量或单条），统一数值类型（整数/两位小数）、处理日期类型，结果写入 JSON 文件。

### ***evaluation.py***

线下评测器。把待测结果与标准答案做规整化比较（行内容+行序无关），支持题目特判（如 `sql_111` 坐标需带小数），输出评测报告与列表文件。

## **2.3 KnowledgeVerificationAndNegativeConstraint**

历史正确/错误结果的守护与反向约束、交互式探针代理

### **核心痛点与设计初衷：**

在实际应用中，大模型常陷入两大困境：一是“重蹈覆辙”，容易对某些特定的逻辑陷阱产生顽固记忆，反复生成相同的错误 SQL 且难以自知；二是“盲目猜想”，面对 Schema 中未明示的隐性业务规则（如特定状态过滤），模型往往只能依靠幻觉进行猜测，缺乏基于真实数据的判断能力，导致生成的 SQL 逻辑偏离实际业务。

### **解决方案：**

如果说上一模块是在学习“如何做对”，本模块则致力于\*\*“防止做错”与“挖掘隐性规则”。首先，充分利用历史错题（Negative Samples）建立错题本



(Error Book)，并引入 HistoryGuard，在推理时实时拦截已知的错误结果，强制规避“犯同样的错误”。其次，针对模型难以理解的隐性业务逻辑（如默认的“活跃”定义、潜规则过滤条件），我们不再让模型盲目幻觉，而是引入 Data Detective Agent 模仿人类分析师“大胆假设、小心求证”的思维路径：主动向数据库发起探测性查询（Probe），利用真实数据反馈来验证或修正对业务逻辑的猜测，最终将这些经过实证的“默认规则”内化为确定的知识。

### ***data\_detective\_agent.py***

严格遵循 PROBE/SOLVE/CONFIRM 三阶段协议，通过真数据验证假设，避免幻觉；并使用 HistoryGuard 将结果与历史正确答案和已知错误结果集比对，强制规避重复错误；通过 Meta-Analyzer 在被拦截时给出突破建议

### ***build\_error\_feedback.py***

从评测结果文件中抽取指定“错题”的完整结果批次，去重后合并进错题本 error\_feedback.json，供日后 HistoryGuard 反向约束

### ***count\_error\_question.py***

将粘贴的“SQL ID-得分”表格文本解析为正确/错误题目列表的小工具脚本

## **2.4 fscot\_generation**

### **从标准题与正确 SQL 反推规范化推理样本（few-shot COT）**

#### **核心痛点与设计初衷：**

传统的 Few-shot 提示通常只提供“问题”与“正确 SQL”的简单映射(QA Pair)。然而，面对复杂的查询，模型往往知其然而不知其所以然，难以捕捉从自然语言到 SQL 的中间推理跳跃。此外，检索 Few-shot 时若仅依赖问题文本的字面相似度，极易受到具体数值（如日期、金额）的干扰，或忽略了业务场景和表结构深层的一致性，导致检索出的示例“形似神不似”，误导模型。

#### **解决方案：**

针对传统 Few-shot 仅提供简单问答对导致模型难以掌握推理路径，且检索易受具体数值干扰的痛点，本模块构建了“规范化思维链反推+三路多维检索”的高级范式。首先，利用 generate\_fewshot\_cot.py 对标准 Gold SQL 进行反向工程（Reverse Reasoning），强制模型将最终代码拆解为包含 Plan（规划）、Divide（分治）、CoT（推导）的结构化推理步骤，构建出“不仅有答案，更有解题思路”的高质量样本库。在此基础上，我们摒弃单一文本检索，实施“三路多维相似度检索”策略：通过脱敏问题相似度（Masked Question，屏蔽具体时间数字干扰以聚焦句式结构）、知识逻辑相似度（Knowledge，对齐底层业务规则）以及表上下文相似度（Table List，锁定相似 Schema 环境），从语义、逻辑、环境三个维度精准锚定最优样本，确保检索到的 Few-shot 示例能真正引导模型实现高

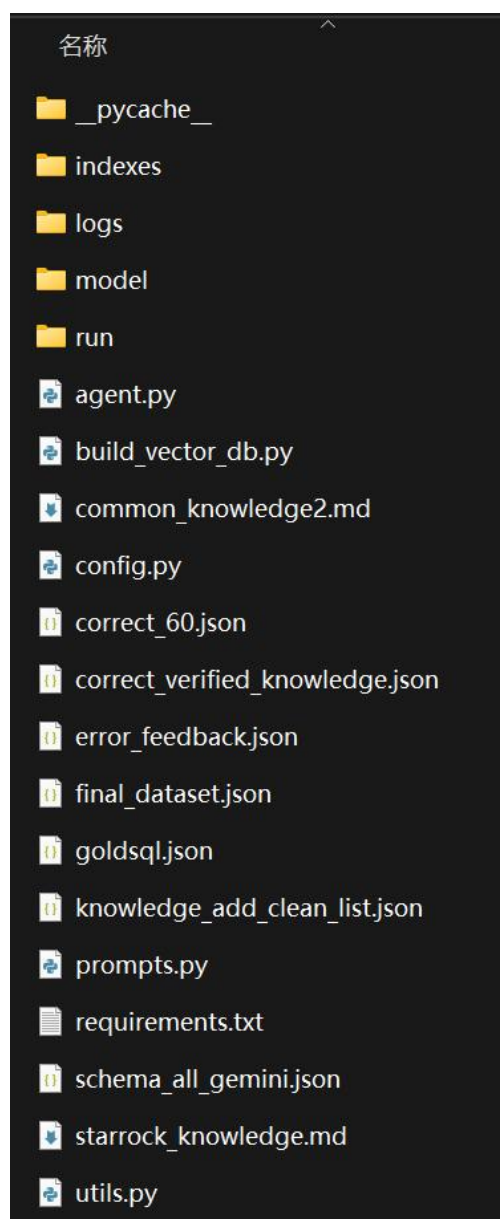
准确率的迁移推理。

### ***generate\_fewshot\_cot.py***

面向给定 sql\_id 集合，使用“问题+正确 SQL+汇总知识”反推规范化 reasoning JSON（包含 plan、divide、cot 三段），用于搭建 few-shot 训练/提示集；支持预览和更新 goldsql.json。

## **3. Pipeline**

文件树：



*build\_vector\_db.py*

从 goldsql.json 生成 Few-shot 检索索引（问句向量+sql\_id 元数据），仅索引 golden\_sql=true 的样本

*utils.py*

支撑工具箱，涵盖文本处理、Embedding 与 FAISS、数据库执行与标准化、日志、SQL 提取

*prompts.py*

定义多种 SQL 生成与修复的 SystemPrompt，按策略拼装对话消息；包含 StarRocks 方言规则与自检清单

*config.py*

集中管理路径、模型/LLM、数据库、检索阈值与阶段产物路径，供整条管线复用

*agent.py*

Text-to-SQL 运行时主控引擎，负责动态 Few-shot 检索与多策略并发生成；集成 StarRocks 物理执行验证与自动修复闭环，最终基于执行结果哈希一致性投票或 LLM 逻辑裁决确定最优 SQL。

4. 答题思路-差异化多路生成策略

我们并行启动 4 个 Agent，每个 Agent 拥有独特的 Prompt 策略和 Schema 视图，以最大化候选答案的多样性。

| 候选者 Agent                     | 核心策略<br>(PromptStrategy)            | 输入视图扰动<br>(InputView)       | 设计意图与核心作用                                                           |
|-------------------------------|-------------------------------------|-----------------------------|---------------------------------------------------------------------|
| Candidate 1<br><br>Standard   | 基准直觉<br><br>直接翻译，不做过度推理。            | 原始 Schema<br><br>(保持原序)     | 基准线(Baseline)<br><br>利用 LLM 的预训练直觉。响应速度快，对于简单、无歧义的问题，通常能给出最直接的正确答案。 |
| Candidate 2<br><br>Schema-CoT | 精准映射(思维链)<br><br>Reasoning->Columns | 列乱序<br><br>(Column Shuffle) | Schema Linking 与<br>值对齐                                             |



|                                                |                                                                  |                                      |                                                                                                                                                       |
|------------------------------------------------|------------------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | -> <b>Filters</b> ->SQL                                          |                                      | <p><b>1.列乱序:</b> 强迫模型全文扫描列描述, 打破对列位置的依赖。</p> <p><b>2. 显式输出</b><br/> <b>Columns/Filters:</b> 强迫模型进行“查字典”确认列名, 并检查过滤条件的值格式。<b>极大减少幻觉列和值格式错误。</b></p>    |
| <p><b>Candidate 3</b></p> <p><b>Divide</b></p> | <p>结构化分治</p> <p>Sub-task1 -&gt; Sub-SQL<br/>-&gt; Assembly</p>   | <p>表乱序</p> <p>(Table Shuffle)</p>    | <p>双重抗干扰与逻辑拆解</p> <p><b>1. 表乱序:</b> 打破模型对 Schema 表顺序的偏见(Positional Bias), 防止模型盲目使用排在前面的表。</p> <p><b>2. 分治思维:</b> 将嵌套/多步逻辑拆解为模块化子查询, 专攻长难句和复杂嵌套逻辑。</p> |
| <p><b>Candidate 4</b></p> <p><b>Plan</b></p>   | <p>执行计划</p> <p>Scan -&gt; Filter -&gt; Join<br/>-&gt; Output</p> | <p>原始 Schema</p> <p>(侧重 Join 路径)</p> | <p>查询优化器视角</p> <p>模仿数据库优化器, 先规划路径再写代码。重点解决<b>多表关联(Join Path)</b>的正确性, 防止笛卡尔积或关联路径错误。</p>                                                              |

生成的 SQL 代码必须经过实战检验。此阶段包含一个自动化的调试循环。

#### 4.1 沙箱执行

将候选 SQL 提交至真实的 StarRocks 数据库执行。

### 4.1.1 错题熔断与智能修复

我们在这里使用了类似人体免疫机制的想法，利用历史经验防止重复犯错，并具备针对性的自我修复能力。

**HistoryGuard:** 基于指纹的错题熔断

代码（WrongResultGuard 类）实现了一个轻量级的哨兵。它不依赖 SQL 文本比对，因为写法可以千变万化嘛，而是对执行结果集生成无序指纹签名（Batch Signature）。

机制：在投票前，系统会实时计算当前 SQL 执行结果的指纹。一旦命中错题本（error\_feedback.json）中的已知错误指纹，该候选 SQL 会被物理剔除。这相当于给系统打了疫苗，从根源上阻断了已知的错误逻辑复现。

**Super Fixer:** 上下文感知的动态修复

修复代理（build\_sql\_fix\_messages\_super）不再是通用请修复，而是基于错误类型动态路由到特定的 Few-shot 修复策略：

**Syntax/Column Error:** 注入 Schema 映射修正的 Few-shot，解决幻觉列名问题。

**Empty Result(0 rows):** 我们认为这是逻辑隐患的高发区。代码会自动注入“检查硬性过滤条件”的提示，防止因条件过严（如 ID='001'vsID=1）导致的数据遗漏。

**Logic Trap:** 代码显式检查是否错误关联了维度表（dim\*），防止内连接（Inner Join）导致的数据过滤风险。

### 4.1.2 基于结果哈希的一致性投票(Result-Consistency Voting)

代码（smart\_sql\_selection）实现了分级决策逻辑：

**一致性收敛:** 若 4 个 Agent 中有 3 个以上跑出了相同的 Result Hash，系统判定为高置信度，直接采纳。

**僵局处理:** 当出现 2vs2 或 1vs1vs1vs1 的分歧时，系统不会随机选择，而是触发高级裁判。

**LLM Judge:** 校验业务规则

当投票失效时，Chief Data Scientist 角色介入（llm\_judge\_selection）。

亮点：裁判不仅看 SQL，更看执行结果数据（Candidates Execution Result）。提示词（JUDGE\_SYSTEM\_PROMPT）包含了一份硬性业务清单（Checklist）：

隐性规则审查：检查是否遗漏了 saccounttype='-100'或 platid=255 等平台级过滤条件。

非空倾向原则：在逻辑看似都合理的情况下，裁判优先倾向于有数据返回的结果（规避空结果陷阱）。

格式对齐：检查日期函数与 Schema 定义的匹配度（String vs Int）。

